

SQL Teaching Exercises: Students, Modules & Lecturers

SQLite exercises – schema design, joins, and foreign keys

PUBLISHED

March 27, 2026

How to use this document. This document lays out some solutions to the questions raised in the slides on RDBMS. I am assuming that you will run code by opening `sqlite3` as follows:

```
sqlite3
```

Then paste the setup script in Section 2 (setup) before running any queries.

1 Full Schema

Note that our schema can be viewed as follows:

Lecturer —< Module —< Enrolment >— Student
(Lec_ID) (Mod_code) (Stu_ID, (Stu_ID)
 Mod_code)

- One **Lecturer** teaches many **Modules**
- One **Module** appears in many **Enrolments**
- One **Student** appears in many **Enrolments**
- **Enrolment** is a *junction table* resolving the many-to-many relationship between Students and Modules

2 Setup: Build and Populate the Database

Paste this entire block into `sqlite3` to create all four tables and insert the sample data.

```
-- Drop everything cleanly if re-running
DROP TABLE IF EXISTS Enrolment;
DROP TABLE IF EXISTS Module;
DROP TABLE IF EXISTS Student;
DROP TABLE IF EXISTS Lecturer;

-- Lecturer
CREATE TABLE Lecturer (
  Lec_ID   TEXT NOT NULL PRIMARY KEY,
```

```
Lec_name TEXT,
Lec_dept TEXT
);

-- Module: each module optionally assigned to one lecturer
CREATE TABLE Module (
  Mod_code TEXT NOT NULL PRIMARY KEY,
  Mod_name TEXT,
  Lec_ID TEXT,
  FOREIGN KEY (Lec_ID) REFERENCES Lecturer(Lec_ID)
);

-- Student
CREATE TABLE Student (
  Stu_ID TEXT NOT NULL PRIMARY KEY,
  Stu_name TEXT,
  Stu_year INTEGER
);

-- Enrolment: junction table linking Students to Modules
-- Composite primary key prevents duplicate enrolments
CREATE TABLE Enrolment (
  Stu_ID TEXT NOT NULL,
  Mod_code TEXT NOT NULL,
  PRIMARY KEY (Stu_ID, Mod_code),
  FOREIGN KEY (Stu_ID) REFERENCES Student(Stu_ID),
  FOREIGN KEY (Mod_code) REFERENCES Module(Mod_code)
);

-- Lecturers
INSERT INTO Lecturer VALUES ('L211', 'Abdul', 'Computer Science');
INSERT INTO Lecturer VALUES ('L555', 'Sarah', 'Economics');
INSERT INTO Lecturer VALUES ('L179', 'Amy', 'Physics');
INSERT INTO Lecturer VALUES ('L234', 'Lou', 'Psychology');
INSERT INTO Lecturer VALUES ('R101', 'Marco', 'Social Science');

-- Modules (U01 Dissertation has no lecturer assigned)
INSERT INTO Module VALUES ('C01', 'Intro to CS', 'L211');
INSERT INTO Module VALUES ('C02', 'DBMS', 'L211');
INSERT INTO Module VALUES ('C22', 'Data Science', 'L179');
INSERT INTO Module VALUES ('C39', 'Computer Networks', 'L179');
INSERT INTO Module VALUES ('E15', 'Econometrics', 'L555');
INSERT INTO Module VALUES ('U01', 'Dissertation', NULL);

-- Students
INSERT INTO Student VALUES ('S001', 'Alice', 1);
INSERT INTO Student VALUES ('S002', 'Bob', 2);
INSERT INTO Student VALUES ('S003', 'Carol', 1);
```

```
INSERT INTO Student VALUES ('S004', 'David', 3);
INSERT INTO Student VALUES ('S005', 'Eve', 2);
INSERT INTO Student VALUES ('S006', 'Frank', 3);

-- Enrolments
INSERT INTO Enrolment VALUES ('S001', 'C01');
INSERT INTO Enrolment VALUES ('S001', 'C02');
INSERT INTO Enrolment VALUES ('S001', 'E15');
INSERT INTO Enrolment VALUES ('S002', 'C01');
INSERT INTO Enrolment VALUES ('S002', 'C22');
INSERT INTO Enrolment VALUES ('S003', 'C01');
INSERT INTO Enrolment VALUES ('S003', 'C02');
INSERT INTO Enrolment VALUES ('S003', 'C39');
INSERT INTO Enrolment VALUES ('S004', 'E15');
INSERT INTO Enrolment VALUES ('S004', 'C22');
INSERT INTO Enrolment VALUES ('S004', 'C39');
INSERT INTO Enrolment VALUES ('S005', 'C02');
INSERT INTO Enrolment VALUES ('S005', 'E15');
INSERT INTO Enrolment VALUES ('S006', 'C01');
INSERT INTO Enrolment VALUES ('S006', 'C22');
INSERT INTO Enrolment VALUES ('S006', 'U01');
```

3 Queries

3.1 Which students take which modules?

A three-way join across **Enrolment**, **Student**, and **Module**.

```
SELECT s.Stu_name,
       m.Mod_name
FROM   Enrolment e
JOIN   Student s ON e.Stu_ID = s.Stu_ID
JOIN   Module m ON e.Mod_code = m.Mod_code
ORDER BY s.Stu_name, m.Mod_name;
```

To also see the lecturer, extend the join. Frank is enrolled in **U01 Dissertation** which has **Lec_ID = NULL** – no lecturer is assigned. Depending on which join type you use, his Dissertation row is either dropped or kept:

```
-- INNER JOIN: Frank's Dissertation row is silently dropped
SELECT s.Stu_name, m.Mod_name, l.Lec_name
FROM   Enrolment e
JOIN   Student s ON e.Stu_ID = s.Stu_ID
JOIN   Module m ON e.Mod_code = m.Mod_code
JOIN   Lecturer l ON m.Lec_ID = l.Lec_ID
```

```
ORDER BY s.Stu_name;

-- LEFT JOIN: Frank's Dissertation row is kept, Lec_name shows NULL
SELECT s.Stu_name, m.Mod_name, l.Lec_name
FROM   Enrolment e
JOIN   Student   s ON e.Stu_ID   = s.Stu_ID
JOIN   Module     m ON e.Mod_code = m.Mod_code
LEFT JOIN Lecturer l ON m.Lec_ID  = l.Lec_ID
ORDER BY s.Stu_name;
```

Rule of thumb: use **LEFT JOIN** whenever the foreign key is optional, i.e. when **NULL** is a valid value in your schema.

3.2 How many modules does each student take?

COUNT with **GROUP BY**: one row per student.

```
SELECT s.Stu_name,
       COUNT(e.Mod_code) AS modules_taken
FROM   Student s
LEFT JOIN Enrolment e ON s.Stu_ID = e.Stu_ID
GROUP BY s.Stu_ID, s.Stu_name
ORDER BY modules_taken DESC;
```

LEFT JOIN here means a student with *no* enrolments still appears with a count of 0 rather than being excluded entirely.

3.3 Which lecturers teach the most students?

We need **COUNT(DISTINCT e.Stu_ID)** rather than plain **COUNT(*)**. Without **DISTINCT**, a student taking two of Abdul's modules would be counted twice once per module. You may wish to run this counting totals including non-distinct matches to see differences.

```
SELECT l.Lec_name,
       COUNT(DISTINCT e.Stu_ID) AS students_taught
FROM   Lecturer l
JOIN   Module     m ON l.Lec_ID  = m.Lec_ID
JOIN   Enrolment e ON m.Mod_code = e.Mod_code
GROUP BY l.Lec_ID, l.Lec_name
ORDER BY students_taught DESC;
```

Marco (R101) does not appear — he has no modules, so the **JOIN** to **Module** produces no rows for him. Try changing the first **JOIN** to a **LEFT JOIN** to see what happens to his count.

3.4 Which lecturers have no students?

We can use a subquery for this... A subquery finds all lecturer IDs connected to at least one enrolment. **NOT IN** returns everyone outside that set.

```
SELECT Lec_name
FROM   Lecturer
WHERE  Lec_ID NOT IN (
    SELECT DISTINCT m.Lec_ID
    FROM   Module    m
    JOIN   Enrolment e ON m.Mod_code = e.Mod_code
    WHERE  m.Lec_ID IS NOT NULL
);
```

The **WHERE m.Lec_ID IS NOT NULL** guard is essential. If any **Lec_ID** in the subquery result is **NULL**, then **NOT IN (... , NULL, ...)** returns no rows at all: a common SQLite trap.

4 Foreign Keys: Deleting a Lecturer with Enrolments

4.1 SQLite foreign keys are OFF by default

Unlike most databases, SQLite does not enforce foreign key constraints unless you turn them on explicitly **and** the setting resets every time you open a new connection.

```
-- Check current state: 0 = off, 1 = on
PRAGMA foreign_keys;
```

4.2 Deleting without enforcement (default behaviour)

With foreign keys off, SQLite will silently delete a lecturer whose **Lec_ID** is still referenced in **Module**. As discussed in class, the result is *orphaned rows*: modules pointing to a lecturer who no longer exists.

```
-- This succeeds without any error
DELETE FROM Lecturer WHERE Lec_ID = 'L211';

-- Abdul is gone, but C01 and C02 still reference L211
SELECT * FROM Module WHERE Lec_ID = 'L211';
```

This is silent data corruption. Any subsequent join between **Module** and **Lecturer** will either silently drop C01/C02 (INNER JOIN) or return NULL for all lecturer columns (LEFT JOIN).

Restore Abdul before continuing:

```
INSERT INTO Lecturer VALUES ('L211', 'Abdul', 'Computer Science');
```

4.3 Deleting with enforcement enabled

```
-- Turn foreign key enforcement on for this session
PRAGMA foreign_keys = ON;

-- The same delete is now blocked
DELETE FROM Lecturer WHERE Lec_ID = 'L211';
-- Error: FOREIGN KEY constraint failed
```

`PRAGMA foreign_keys = ON` must be set every time you open SQLite. It does not persist between sessions.

4.4 How to delete correctly

Resolve the dependencies first. Two options:

Option A – Unassign the modules (set `Lec_ID` to `NULL`):

```
UPDATE Module SET Lec_ID = NULL WHERE Lec_ID = 'L211';
DELETE FROM Lecturer WHERE Lec_ID = 'L211';
```

Option B – Delete the modules and their enrolments first:

```
-- Remove enrolments for Abdul's modules
DELETE FROM Enrolment
WHERE Mod_code IN (SELECT Mod_code FROM Module WHERE Lec_ID = 'L211');

-- Remove the modules themselves
DELETE FROM Module WHERE Lec_ID = 'L211';

-- Now the lecturer delete succeeds
DELETE FROM Lecturer WHERE Lec_ID = 'L211';
```

SQLite also supports **ON DELETE CASCADE** declared in the schema, which handles the cascade automatically, but only when **PRAGMA foreign_keys = ON** is active.